Member-only story

Esp32 | IoT | Espnow | Sensors | Microcontrollers

IoT without a Router

ESP32s have a special protocol for IoT

Leslie Foster Following 14 min read · 7 hours ago

1

WiFi is ubiquitous in many countries. It is in homes, offices, shops, and some municipalities even offer WiFi connectivity. We often will see a sign somewhere on the premises telling us what the WiFi SSID and Password are, so we can connect our device. But what about “connectionless”? Well, it means the router is not needed. Two devices can interact directly without going through a dedicated network box somewhere. That can be vital if you place them somewhere that has no other WiFi.

Background on Microcontrollers

Microcontrollers like the ESP32s are made to be deployed somewhere and for the most part left alone to do their jobs. Sometimes they can be left unattended (or as attendants) for years, even running on small batteries. They are system-on-a-chip implementations that have not only what it takes

to be a computer, but also “peripherals” of varying degrees of complexity, from simple 1/0 pins all the way up to these radios. Their on-chip hardware is connected to the rest of the world through pins. The Xiao layout is rather deceptive in that it has several “pads” on it for various things, *plus* pins, plus its USB connector, buttons, and one or more LEDs. But in the simplest terms, there is a chip with pins that is soldered onto its development board.

Microcontrollers typically require code to be “flashed” onto them after cross compiling on a larger, more general-purpose system. These are no different, and the development board packages covered here avoid any need to purchase extra equipment.

As a 32-bit offering, these ESPs have a much more layered firmware approach than their older 8-bit cousins. The ESPs (among others) have the FreeRTOS Real Time OS baked right in their code, if you use Espressif’s tools.

ESP32

In other articles in this ‘series’, we have discussed the ESP32 family of microcontrollers. Their 32-bit, 100MHz+ processors are often bundled (as in the Xiao systems from Seeed Studios) with significant amounts of memory compared to other MCU counterparts, and importantly, on-chip radio hardware, as was used in the Wifi Strength Meter and in Sending Data to the Cloud. The ESP32C6 is a prime example, being “radio rich”, but most ESP32 offerings have at least Bluetooth Low Energy and WiFi. The Xiao bundles have these chips soldered onto postage-stamp-sized printed circuit boards (PCBs) with a USB connector for programming and optional communication. As we saw in the earlier articles, with all that attention to wireless, these tiny devices can do things like talk to web sites and AWS, act as router repeaters, and establish mini WiFi hotspots. They can even host websites. And since

they are from the Esspressif Systems Corporation, they support a proprietary protocol called ESP-NOW.

ESP-NOW is a connectionless protocol, in that you do not need to use your WiFi access point to allow two of the systems to talk to each other. You would not be able to use ESP-NOW to directly send data to the cloud, but you could setup a peer network and let several of them talk to another computer that does connect to the cloud. Indeed, you could arrange for one of a set of ESP32s to take on that role, to relay for other ESP32s. The protocol can be used to establish linkages from peer to peer, given known or discoverable MAC addresses (these are like the MAC addresses for any other device, such as your home computer). It can also be used to send and receive broadcast messages. Of course, broadcast is less secure and less specific, but it will be described here because it is simpler, and a good way to get started with the least fuss. Do be aware that many of these Xiao boards are sold without the headers (pins) soldered on, and pins are required for breadboard use.

Why this is not your last stop

What we will see here is simplified code. It has no encryption in flight, pushes data to nonspecific receivers, and has no consideration of things like Over the Air updates (OTA). If you were to do this for production use, you would not want any sensitive data being sent this way. And if you deployed something with such a nice radio system, you would want to be able to leverage that capability to perform remote updates as needed. But this might serve as a gentle (enough) introduction to the technology.

The Project

To get some utility out of the communication method, we will implement a simple weather station that collects and transmits temperature and

atmospheric pressure from one ESP32 to another. Most or all of the Xiao boards could be used as either endpoint of this project, but we will use an ESP32C6 for the display and an ESP32C3 for the sensor. This is a reasonable choice because the C3 is potentially a much lower power-consumer than the C6, which in turn is powerful enough to accept multiple incoming transmissions, and has more options for radio communication. Both are easily usable with Espressif's proprietary ESP-NOW protocol.

For this project we will use software called "IDF". Unlike Arduino (which would also work), IDF is distributed by the ESP32 vendor: Espressif. It is well supported, and has a fairly extensive community, so it is easy to get out of any bind you may find yourself wandering into. These modern chips are programmed in C (or C++), and when mounted on the PCB like a Xiao board, can be programmed using IDF software, your PC, and a USB cable. On a Windows system, IDF tools come as a combination of a Windows CMD shell configuration and a VS Code extension. You use VS Code as you would on other projects, but with C as the programming language, and including whatever Espressif libraries are required.

Project Usability

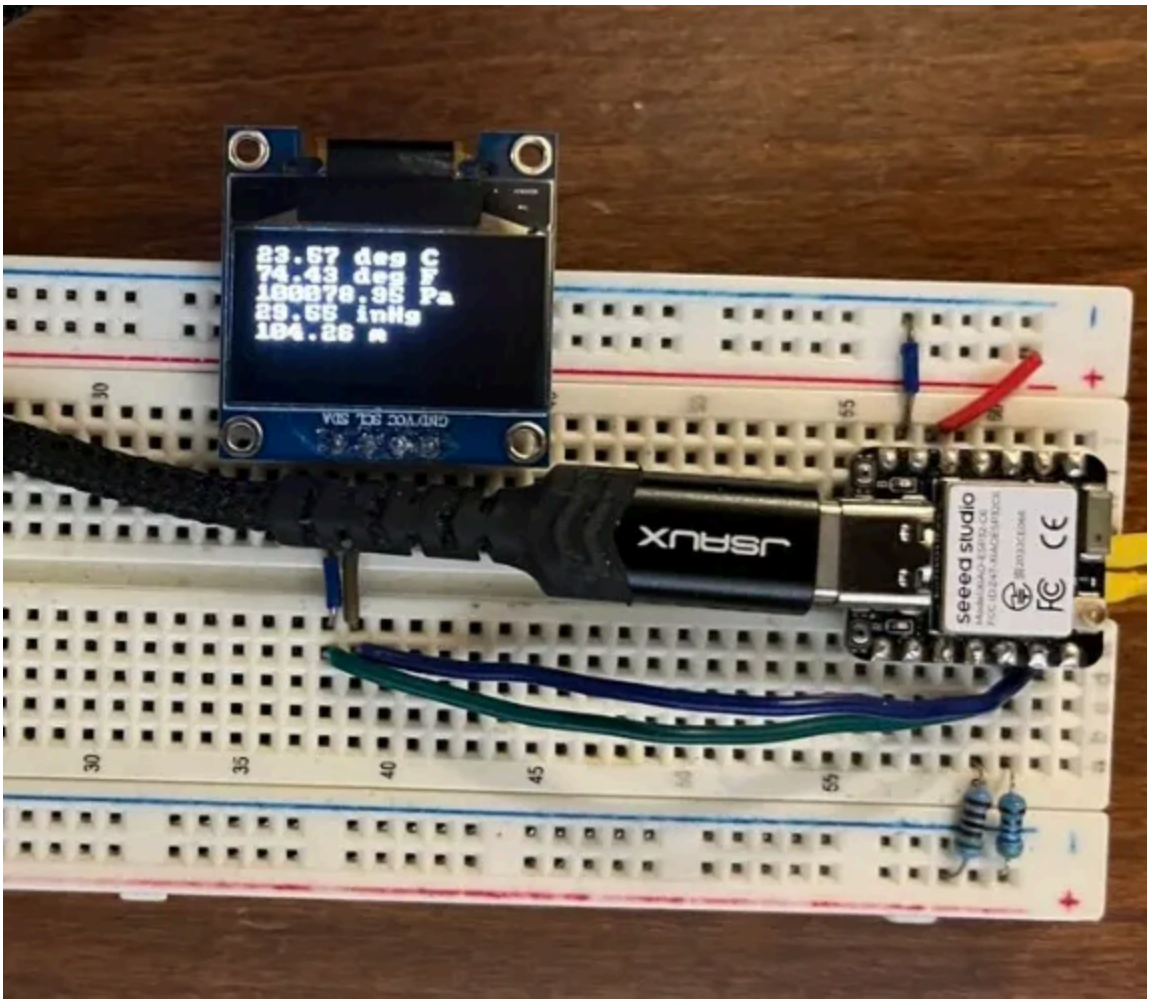
ESP32C3 Xiao's ship with a detached antenna that can be snapped into place. The Xiao ESP32C6 does not come with an antenna, but a similar one can snap right into it as well, as it has the same connector. In past projects, it has been found that the C6 can do nearby communications very easily without an external antenna. In one instance, we used it to detect WiFi signal strength, where it reported effectively whether sufficient strength was available for a much larger piece of equipment. It received about as well as the wall-powered consumer device did. However, the C3 really does require *its* antenna to get any distance at all. And, the C6 does benefit, if we want *maximum* distance. It was found that with an antenna on the C3, and none

on the C6, the devices could communicate even if one was one floor down from the other, and maybe 20 feet away. Adding an antenna to both devices resulted in transmissions succeeding between three separate floors — that's through two layers of flooring, and any walls that happened to be between them. It will very likely work through the outer wall of a house if you are thinking of making a real weather station. We will use breadboards, here. Do not use a breadboard outdoors, for obvious reasons.

Required Materials

Being about communication, this project can be divided into two parts. One is the display part (the base) and the other collects data (the sensor).

The Base



The ESP32C6 Serves as the base for the weather station, displaying data on an SSD1306

We reuse parts of a project discussed earlier here, where we exploited a small LSD display called SSD1306, which connects to the ESP32 using I2C, and also used I2C to gather sensor data.

This is the parts list. All prices are in USD (see further cost explanations below):

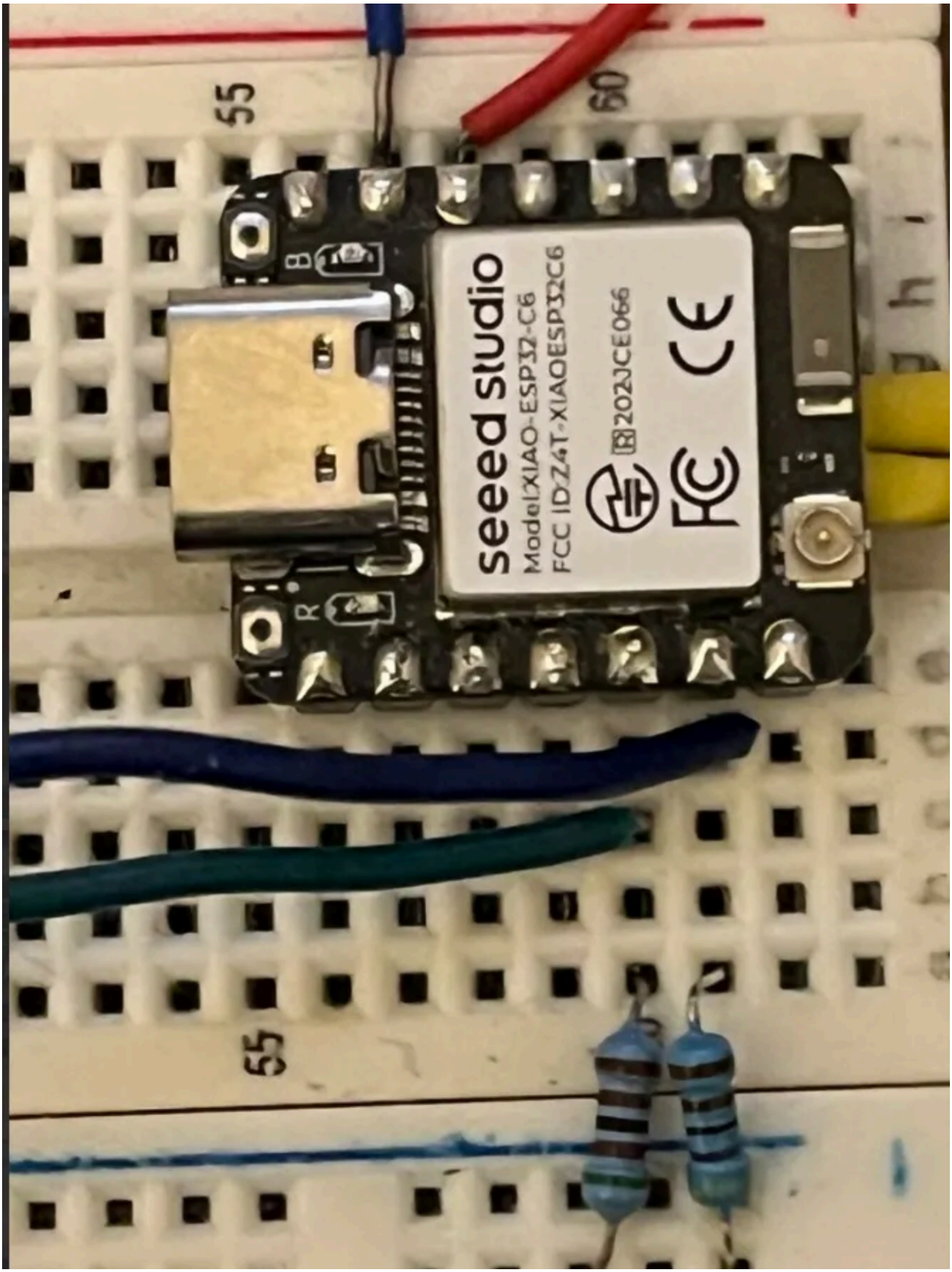
- ESP32C6 Xiao. Others are usable, even the C3 if you have two, but pins must be soldered on to make it breadboard-compatible. If you do not use an ESP32C6, you will have to change the code to account for different SCL and SDA pin assignments. $5.20 USD to $13 USD.
- Breadboard
- PC for programming with ESP-IDF installed
- VS Code with ESP-IDF extension installed
- USB cable for programming; doubles as power supply
- SSD1306. These may be had for about 3 for $10 USD through Amazon.
- Two 2KΩ to 10kΩ resistors
- Wires (whatever works well for a breadboard; an electronics kit will have

sufficient.

Base Wiring

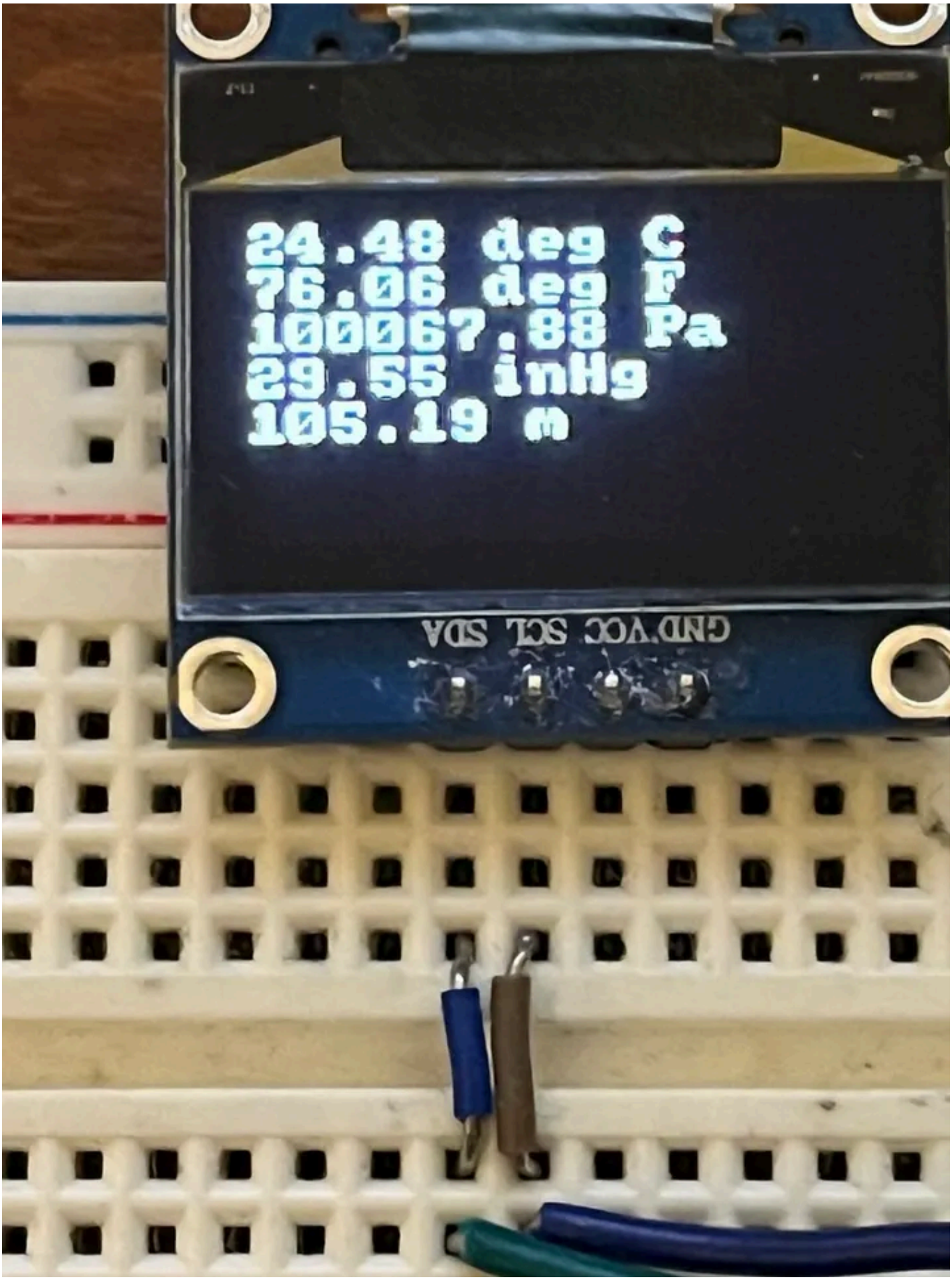
We need the pullups, as always, to give the twin I2C conductors default *high* values. They must be pulled up to Vcc level. Although 5V is supplied by the USB cable, the Xiao board reduces that effectively to 3.3V, which is correct for the ESP32s.

Here, the pullup is right where the I2C wires connect to the Xiao. The other ends of those wires connect to the display.



Two resistors pulling up the I2C conductors to 3.3V

As can be seen above, the blue cable is going to the 2nd pin from the right (the small round gold-ish connector is for the antenna; we will refer to this as the antenna side), while the green goes to the 3rd pin from the right. You can use the USB connector (at left) as a guide while placing the resistors and connecting these wires.



The green is for SDA (data); the blue is for SCL (clock)

These two wires are clock and data. As seen above, they connect to thus-labeled pins on the display device. So, this takes care of the display. But as you can see in the images, there is no sensor on this board at all.

The Sensor

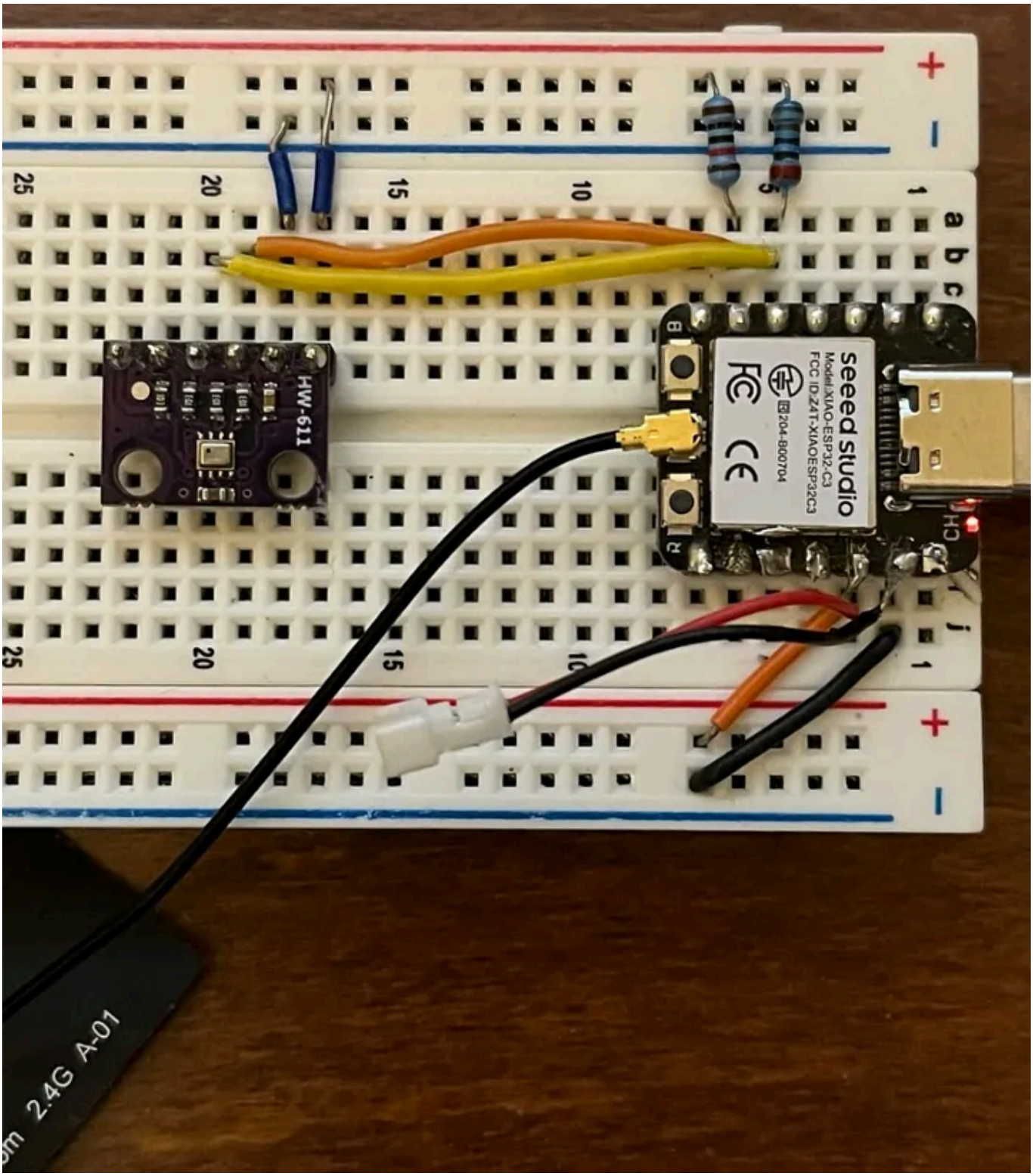
The earlier article (again, here) already describes how to use the sensor. The BMP280 collects atmospheric pressure and temperature. While pressure is not as common a measurement as humidity, these inexpensive sensors connect using I2C, unlike the DHT's, and we are usually more interested in temperature than things like pressure *or* humidity, anyway. It is up to you whether to work with a humidity sensor instead, or even to just send button presses back and forth. But, for this project, we will proceed with the BMP sensors.

You will therefore need:

- ESP32C3 (a second C6 would work, but the pin assignments are different in the code. If you use a C6, the code in the other article should be a good reference). These are $20 USD for a set of three. See further explanations below.
- BMP280 sensor. These may be purchased in various set sizes (3 to 10), averaging less than $1.50 USD each.
- Two 2kΩ to 10kΩ resistors
- Breadboard
- PC for programming with ESP-IDF installed
- VS Code with ESP-IDF extension.
- USB cable

This article uses an ESP32C6 on one side and an ESP32C3 on the other, but since they are often sold in sets for a bulk discount, it may be cheaper to use two of the same kind — if, as mentioned, you change the pin references in the code.

Sensor Wiring



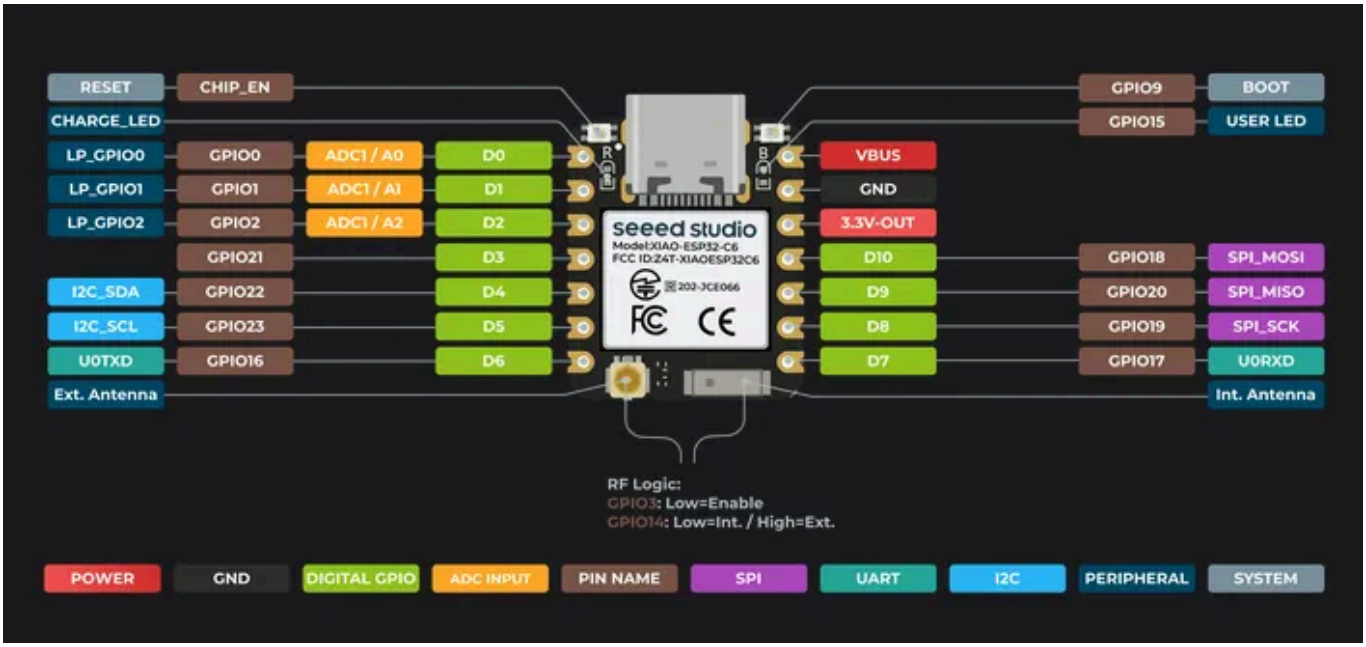
Note the antenna on the C3; note the pullups on the I2C cables

This wiring on this one is also quite simple. Just be sure to get the pullups situated properly. Right-at-the-Xiao connections seemed to work well. Here, the SDA line also runs from the third pin from the “antenna side”, and the SCL line runs from the second pin from the antenna side. This C3 has its

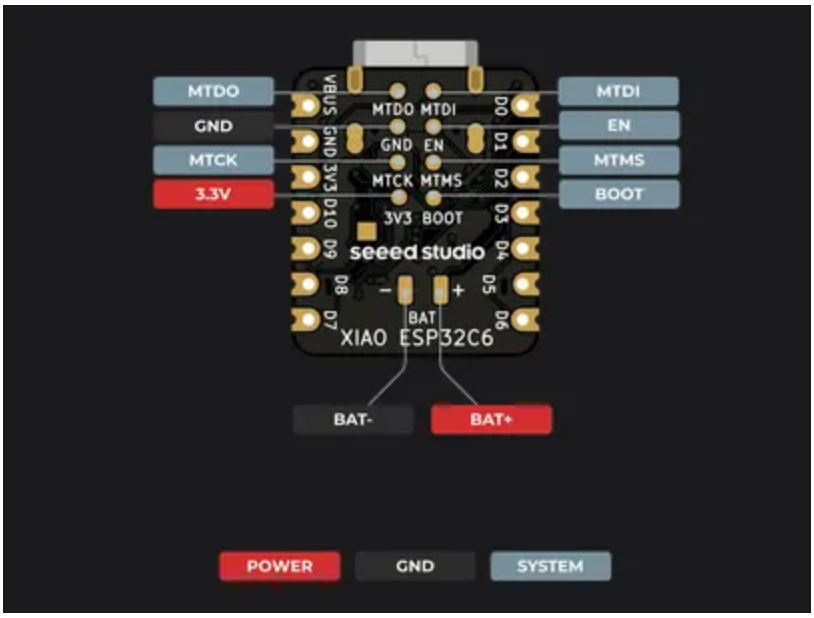
antenna connected. That's the black wire running to the flat object lying partly under the breadboard.

There is a power and a ground wire also running to the sensor. Not shown: it is necessary to run wires *across* the breadboard to ensure power and ground run to both rails of the board.

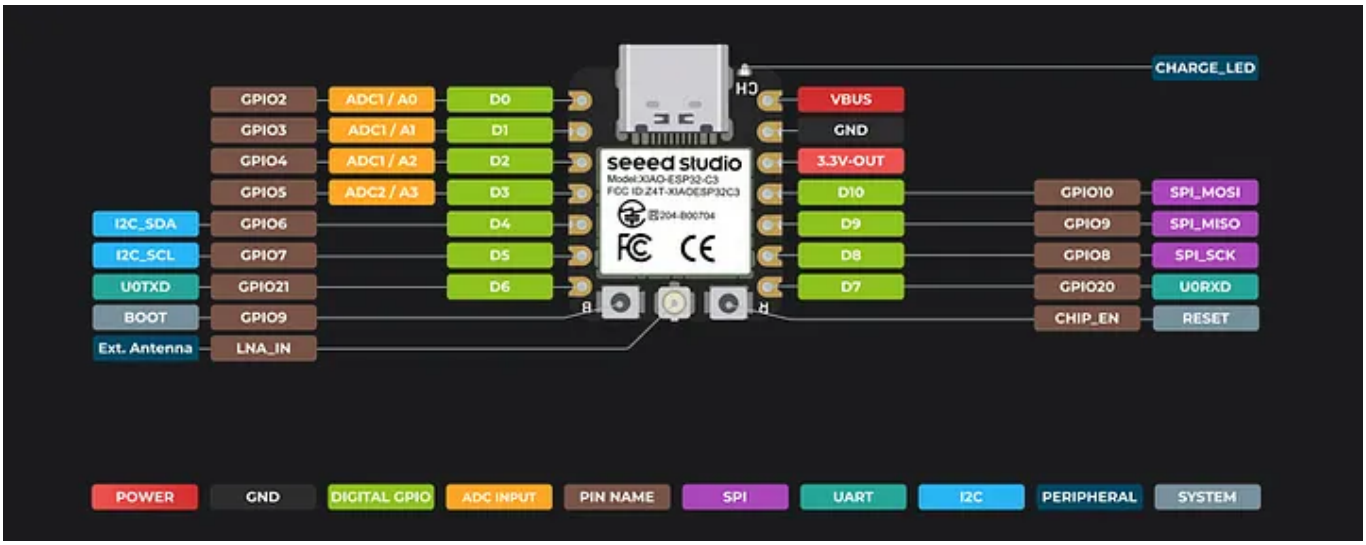
Both of these communication participants could be placed on the very same breadboard, as neither of the circuits take much room. Sharing a common ground, they can even have separate power supplies. But then, the entire utility of wireless is lost to us. Using separate breadboards, we can move them far apart.



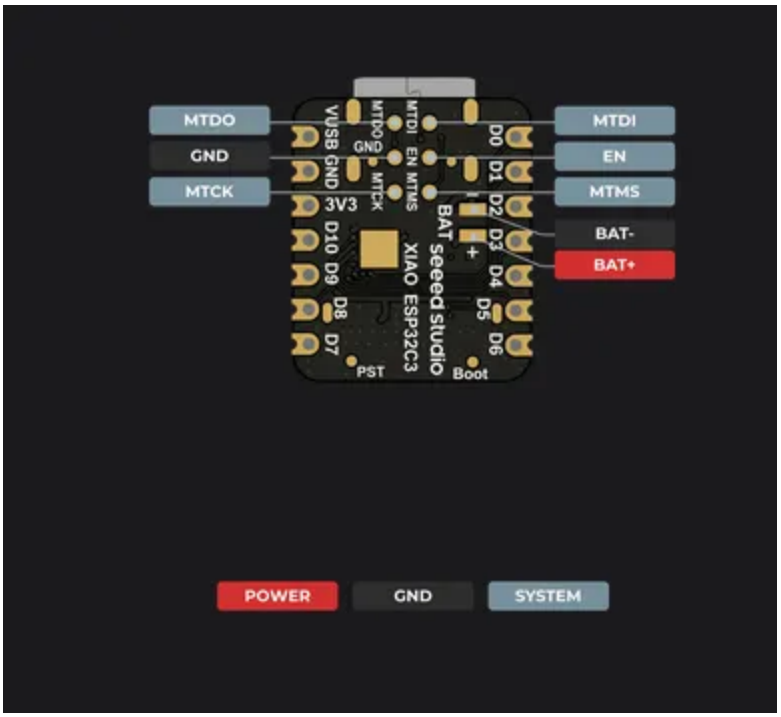
ESP32-C6 Pinout from Getting Started Guide (top)



ESP32-C6 underside. Battery pads are under the antenna end



ESP32-C3 Pinout from the Getting Started Guide (top)



ESP32-C3 Pinout [bottom]. Note the battery pad placement. The battery pads are small, flat contacts on the printed circuit board

Power

Other than the USB cables, the recommended way of powering Xiao boards is by rechargeable 3.7V batteries. That's higher than the 3.3V upper end, but the onboard power regulator ensures a proper flow, and starting with 3.7 means they can discharge quite a bit before having too low output. The regulator should cut off around 3.0V. The Xiao's, if you get these batteries soldered on properly, can be used to recharge the batteries and avoid the need for a separate charger. It should be noted that certain batteries (available from Maker Focus, for example) are specially made for boards like the Xiaos, and are forgiving about things like leaving them connected too long. They could be more expensive than some, but for safety and reliability, they are a good investment.

It may be possible to power these MCUs using the Xiao's 3V3 and Ground *pins*. But the pins are considered an output supply, so it is not recommended.

If you do, try to use an appropriate battery voltage. Please do not try 5V. Even the 3.7V batteries are probably too high.

Connecting batteries to the Xiao pads can be tricky, because you are soldering onto two tiny areas, and because it is hard to keep the wire strands of the connector together properly. Some ideas about that are discussed here, for those who are interested. You may find that the C6 is easier to solder a battery onto in the proper way. The C3 is trickier, because its pads are aligned sideways and sitting pretty close to where the header pins' plastic holder would go. It can be difficult to get a soldering iron in there, even with the "pulled pin" technique discussed in the other article. You can see how close things are in the pinout images above.

Of course, if you have two USB power supplies, you can hook them up as you please.

The Code

In many of these articles, the code is simply pasted in, but for this two-part project, it is not practical. Interfacing with these two separate peripherals required libraries, plus there needs to be separate projects just to avoid confusion. So, instead, this code will be distributed through github. This is the repo: https://github.com/lesfoster/esp32_publications.git, which includes both of these projects.

This code sends broadcast packets to any other ESP-NOW device that may be listening. When messages arrive, they consist of the full text to be displayed. A tighter approach might be to send the BMP register contents directly, or even just converted values. BMP280 sensor interpretation is not a trivial process: it requires running algorithms just to get sensible numbers. The sensor has registers containing factory-tuned parameters that must be

applied to each temperature or pressure value in a complex series of calculations, or the results are completely meaningless. So, keeping those calculations close to the attached MCU is best. Even so, labeling might be stripped off to achieve leaner bandwidth usage.

Likewise, these chips can be programmed to *encrypt* messages in flight. This information is not very sensitive, unless you happen to have an air conditioning salesman passing by with some kind of ESP-NOW monitor and they want to pressure (no pun intended) someone into a sale. Not very likely. Still, these ESP chips, and their ESP-NOW software, are amenable to encryption and other security enhancements.

Lastly, we have not attempted to squeeze out the maximum battery life. A battery was used to power the sensor side but lasted less than 24 hours. That sounds disheartening, until we realize that the data are transmitted at five second intervals, which is not only far more frequently than most of us would require, but that the opportunity of putting the ESP32-C3 into a low power “sleep mode” was not even attempted. It could be sleeping most of the time, waking up after minutes (not seconds) to send a packet to a specific endpoint, with encryption, and still use far less power per time period than it does always-awake.

Flashing the Code

Working with ESP32’s Espressif system is like working with any IDE, except for the flashing process. there is a full tutorial on getting started in the references below, and the WiFi strength article goes into some depth on that. However, the steps are basically:

- Install the Espressif IDF software.
- Install Espressif IDF extension in VS Code.

- Create a project and edit (or paste in) the code. For this project, you could just clone the repository and open the folder in VS Code.
- Open the ESP-IDF CMD App, which is a specially configured Windows CMD environment. Alternatives exist for Linux and Mac.
- Change directory to where your project is on disk. There are two projects in the repo that are of interest. oled-esp-now is for the one that has the SSD1306 screen; temperature-pressure-esp-now is for the one with the BMP280 sensor.
- Be sure to have your ESP32 connected over USB (USB-C) cable to your PC, and make note of where the port is. It can be done with Device Manager on Windows.
- At the command line, change to the project directory, set the project's target to the type of ESP you are using ("idf.py set-target esp32c6" or "idf.py set-target esp32c3"). Naturally, you do this separately for the oled-esp-now and temperature-pressure-esp-now projects.
- Run commands like "idf.py build", "idf.py -p PPPP flash", "idf.py -p PPPP monitor", etc., where "PPPP" is whatever port has been assigned to the plugged-in device.

Of course, as you learn and grow (and encounter bugs) you go through another development iteration and flash it again.

MCU Cost and Alternatives

If you have purchased a sensor kit, it may have the BMP-280 or BMP-180 already in it. There are also devices like the *BME*-280, which includes temperature, humidity and atmospheric pressure all in one. However, the code accompanying this article interfaces only with the BMP-280.

ESP32-C6s may be purchased on Amazon at $23 USD for a set of three, or $12–13 each. Other options may be available. In fact, a set of three from DigiKey and Mouser at time of writing was only $15.86, or $6.36 for just one. One individual device included pre-soldered pins. Be warned that *shipping costs* from those outlets can be higher. One strategy might be to order other things at the same time, and spread the shipping cost, rather than pay that for just one item.

The ESP32-C3 Xiaos usually sell for a bit less than the C6. They are available for $20 USD for a set of three on Amazon. As with the C6s, you can check prices and shipping on DigiKey, or Mouser.

Similarly, you can go straight to the Seeedstudio website and order these boards even cheaper, but the shipping cost is still a consideration.

The Xiao boards used here work fine. But there are other layouts for the ESP32 that should also work. If you purchase them for this purpose, you should try to verify first that they support ESP-NOW. Indeed, even one obsolete and very cheap model — the ESP8266 — is said to be ESP-NOW compatible. You may have to make changes to the code linked to this project if trying something very different from the boards presented here.

Conclusions

You can push remote data around using a pair of relatively cheap MCU development boards, without having to be nearby to a WiFi router, or to key in the router's WiFi SSID or password. We used this connection-less capability to remotely monitor a sensor device. Given the caveats mentioned earlier re: security and power budget, you now have enough information to set up an Internet of Things network using modern, 32-bit systems. If the security and power constraints are addressed, you can do a lot more than

just take weather measurements. These 32-bit devices can run AI models of reasonable sophistication, and report back what was done. And if you employ OTA, adjustments can be made over time without costly or time-consuming travel.

What to do next is up to you.

References

- ESP32C6 data sheet
- ESP32C3 data sheet
- ESP32C6 WiFi strength article
- ESP32C6 Sensor article
- Source code https://github.com/lesfoster/esp32_publications.git
- Getting started with ESP32C6
- Getting started with ESP32C3

Esp32 + IoT + Espnow + Sensors + Microcontrollers +

Written by Leslie Foster

120 followers · 8 following

Programmer / Developer / Software Engineer since late 1980s

Following